

Practical 1

Project: Movie Collection Manager

Objective: Work with arrays and objects to manage a collection of movies dynamically.

Tasks:

- Create an array of movie objects, where each object has fields like title, genre, rating, and release year.
- Write functions to:
 - Add a new movie to the collection.
 - List all movies in a specific genre.
 - Find the highest-rated movie in the collection.
 - Use `map()` to create a list of all movie titles.
 - Use `filter()` to find movies released after a specific year.
 - Log the results to the console using template literals.

Screenshots of Code:

```
Practical-1 > Movie manager > JS movieManager.js > ...
1  const movies = [
2    { title: "Inception", genre: "Sci-Fi", rating: 8.8, releaseYear: 2010 },
3    { title: "The Dark Knight", genre: "Action", rating: 9.0, releaseYear: 2008 },
4    { title: "Interstellar", genre: "Sci-Fi", rating: 8.6, releaseYear: 2014 }
5  ];
6
7  const addMovie = (title, genre, rating, releaseYear) => {
8    if (!title || !genre || !rating || !releaseYear) {
9      console.error("❌ Error: All movie fields are required.");
10     return;
11   }
12
13   const newMovie = { title, genre, rating, releaseYear };
14   movies.push(newMovie);
15   console.log(`✅ Movie "${title}" added successfully.`);
16 };
17
18 const listMoviesByGenre = (genre) => {
19   return movies.filter(movie => movie.genre.toLowerCase() === genre.toLowerCase());
20 };
21
22 const findHighestRatedMovie = () => {
23   return movies.reduce((highest, movie) => movie.rating > highest.rating ? movie : highest);
24 };
25
26 const getMovieTitles = () => {
27   return movies.map(movie => movie.title);
28 };
```

```

30  const moviesAfterYear = (year) => {
31      return movies.filter(movie => movie.releaseYear > year);
32  };
33
34  const logMovies = () => {
35      movies.forEach(({ title, genre, rating, releaseYear }) => {
36          console.log(` 🎬 ${title} (${releaseYear}) - ${genre} | Rating: ${rating}`);
37      });
38  };
39
40  addMovie("Tenet", "Sci-Fi", 7.5, 2020);
41  addMovie("Avengers: Endgame", "Action", 8.4, 2019);
42
43  console.log(" 🗨️ Sci-Fi Movies:", listMoviesByGenre("Sci-Fi"));
44  console.log(" 🏆 Highest-Rated Movie:", findHighestRatedMovie());
45  console.log(" 📖 All Movie Titles:", getMovieTitles());
46  console.log(" 📅 Movies Released After 2010:", moviesAfterYear(2010));
47  logMovies();

```

Screenshots of Output:

```

C:\Users\Admin\Documents\full_stack\Practicals\Practical-1\Movie manager>node movieManager.js
✅ Movie "Tenet" added successfully.
✅ Movie "Avengers: Endgame" added successfully.
🗨️ Sci-Fi Movies: [
  {
    title: 'Inception',
    genre: 'Sci-Fi',
    rating: 8.8,
    releaseYear: 2010
  },
  {
    title: 'Interstellar',
    genre: 'Sci-Fi',
    rating: 8.6,
    releaseYear: 2014
  },
  { title: 'Tenet', genre: 'Sci-Fi', rating: 7.5, releaseYear: 2020 }
]
🏆 Highest-Rated Movie: {
  title: 'The Dark Knight',
  genre: 'Action',
  rating: 9,
  releaseYear: 2008
}
📖 All Movie Titles: [
  'Inception',
  'The Dark Knight',

```

```
'Interstellar',
'Tenet',
'Avengers: Endgame'
]
📅 Movies Released After 2010: [
{
  title: 'Interstellar',
  genre: 'Sci-Fi',
  rating: 8.6,
  releaseYear: 2014
},
{ title: 'Tenet', genre: 'Sci-Fi', rating: 7.5, releaseYear: 2020 },
{
  title: 'Avengers: Endgame',
  genre: 'Action',
  rating: 8.4,
  releaseYear: 2019
}
]
📅 Inception (2010) - Sci-Fi | Rating: 8.8
📅 The Dark Knight (2008) - Action | Rating: 9
📅 Interstellar (2014) - Sci-Fi | Rating: 8.6
📅 Tenet (2020) - Sci-Fi | Rating: 7.5
📅 Avengers: Endgame (2019) - Action | Rating: 8.4
```

Conclusion:

In this lab, we explored fundamental JavaScript concepts, including variables, functions, arrays, objects, and ES6+ features. By implementing the Movie Collection Manager, we practiced applying these concepts in a structured and practical manner.

GitHub Link:

<https://github.com/VRAJPATEL-07>

Practical 2

Project: Task Reminder System

Objective: Build a task reminder system that schedules reminders and handles asynchronous delays.

Tasks

- Create an array of task objects, where each task has fields like title, dueTime (in minutes from now), and priority.
- **Write functions to:**
 - Add a new task to the list.
 - Sort tasks by their priority.
 - Display tasks due within a certain timeframe (e.g., next 10 minutes).
 - Simulate sending reminders using setTimeout() based on the task's dueTime.
- Use try...catch to handle errors, such as invalid task data or missing fields.
- Export and import the task management functions using JavaScript modules.

Screenshots of Code:

```
Practical-2 > Task reminder > JS taskManager.js > ...
1  const tasks = [];
2
3  export const addTask = (title, dueTime, priority) => {
4    try {
5      if (!title || typeof title !== "string") throw new Error("Title must be a non-empty string.");
6      if (isNaN(dueTime) || dueTime <= 0) throw new Error("Due time must be a positive number.");
7      if (!["high", "medium", "low"].includes(priority)) throw new Error("Priority must be 'high', 'medium', or 'low'.");
8
9      const newTask = {
10         title,
11         dueTime: new Date(Date.now() + dueTime * 60 * 1000),
12         priority,
13       };
14
15       tasks.push(newTask);
16       console.log(`✅ Task "${title}" added successfully.`);
17     } catch (error) {
18       console.error(`❌ Error adding task: ${error.message}`);
19     }
20   };
21
22   export const sortTasksByPriority = () => {
23     const priorityOrder = { high: 1, medium: 2, low: 3 };
24     return tasks.sort((a, b) => priorityOrder[a.priority] - priorityOrder[b.priority]);
25   };

```

```

27 export const getTasksDueIn = (minutes) => {
28   const now = new Date();
29   const futureTime = new Date(now.getTime() + minutes * 60 * 1000);
30   return tasks.filter(task => task.dueTime >= now && task.dueTime <= futureTime);
31 };
32
33 export const sendReminders = () => {
34   tasks.forEach(task => {
35     const timeUntilDue = task.dueTime - new Date();
36     if (timeUntilDue > 0) {
37       setTimeout(() => {
38         console.log(`🔔 Reminder: Task "${task.title}" is due now!`);
39       }, timeUntilDue);
40     }
41   });
42 };
43
44 export { tasks };

```

```

Practical-2 > Task reminder > JS testTaskManager.js > ...
1  import { addTask, sortTasksByPriority, getTasksDueIn, sendReminders, tasks } from './taskManager.js';
2
3  addTask("Complete MERN Project", 15, "high");
4  addTask("Revise JavaScript", 5, "medium");
5  addTask("Check Emails", 10, "low");
6
7  console.log("📅 Tasks Sorted by Priority:", sortTasksByPriority());
8
9  console.log("🕒 Tasks Due in 10 Minutes:", getTasksDueIn(10));
10
11  sendReminders();
12
13  const maxDueTime = Math.max(...tasks.map(task => task.dueTime - new Date()));
14  setTimeout(() => {
15    console.log("✅ All reminders have been sent. Exiting...");
16    process.exit();
17  }, maxDueTime + 1000);

```

Screenshots of Output:

```

C:\Users\Admin\Documents\full stack\Practicals\Practical-2\Task reminder>node testTaskManager.js
✅ Task "Complete MERN Project" added successfully.
✅ Task "Revise JavaScript" added successfully.
✅ Task "Check Emails" added successfully.
📅 Tasks Sorted by Priority: [
  {
    title: 'Complete MERN Project',
    dueTime: 2025-03-06T13:38:28.351Z,
    priority: 'high'
  },
  {
    title: 'Revise JavaScript',
    dueTime: 2025-03-06T13:28:28.359Z,
    priority: 'medium'
  },
  {
    title: 'Check Emails',
    dueTime: 2025-03-06T13:33:28.359Z,
    priority: 'low'
  }
]
🕒 Tasks Due in 10 Minutes: [
  {
    title: 'Revise JavaScript',
    dueTime: 2025-03-06T13:28:28.359Z,
    priority: 'medium'
  },
]

```

```
{  
  title: 'Check Emails',  
  dueTime: 2025-03-06T13:33:28.359Z,  
  priority: 'low'  
}
```

```
🔔 Reminder: Task "Revise JavaScript" is due now!  
🔔 Reminder: Task "Check Emails" is due now!  
🔔 Reminder: Task "Complete MERN Project" is due now!  
✅ All reminders have been sent. Exiting...
```

Conclusion:

In this project, we successfully built a Task Reminder System that dynamically schedules and manages tasks while handling asynchronous delays using JavaScript modules, `setTimeout()`, and error handling.

GitHub Link:

<https://github.com/VRAJPATEL-07>

Practical 3

Project: System Utility Dashboard

- **Objective:** Create a command-line tool that provides system information.
- **Tasks:**
 1. Use the `os` module to display the OS type, total memory, free memory, and CPU details.
 2. Use the `fs` module to save the system information to a log file.
 3. Use the `path` module to ensure the file is saved in a standardized format (`logs/system-info.txt`).

Screenshots of Code:

```
Practical-3 > system-utility > JS index.js > ...
1  import fs from 'fs';
2  import os from 'os';
3  import path from 'path';
4
5  const getSystemInfo = () => {
6    return {
7      OS_Type: os.type(),
8      Total_Memory: `${(os.totalmem() / 1e9).toFixed(2)} GB`,
9      Free_Memory: `${(os.freemem() / 1e9).toFixed(2)} GB`,
10     CPU_Details: os.cpus().map(cpu => cpu.model),
11     Uptime: `${(os.uptime() / 3600).toFixed(2)} hours`
12   };
13 };
14
15 const saveSystemInfo = () => {
16   const systemInfo = getSystemInfo();
17   const logData = `System Information:\n${JSON.stringify(systemInfo, null, 2)}\n`;
18
19   const logDir = path.join(process.cwd(), 'logs');
20   const logFile = path.join(logDir, 'system-info.txt');
21
22   if (!fs.existsSync(logDir)) {
23     fs.mkdirSync(logDir);
24   }
25
26   fs.writeFileSync(logFile, logData, 'utf8');
27   console.log(`✅ System information saved to: ${logFile}`);
28 };
29
30 console.log("🔄 Fetching system details...");
31 saveSystemInfo();
```

Screenshots of Output:

```
C:\Users\Admin\Documents\full_stack\Practicals\Practical-3\system-utility>node index.js
🔵 Fetching system details...
✅ System information saved to: C:\Users\Admin\Documents\full_stack\Practicals\Practical-3\system-utility\logs\system-info.txt
```

```
System Information:
{
  "OS_Type": "Windows_NT",
  "Total_Memory": "16.80 GB",
  "Free_Memory": "5.02 GB",
  "CPU_Details": [
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz",
    "11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz"
  ],
  "Uptime": "7.42 hours"
}
```

Conclusion:

In this project, we built a command-line system utility tool using Node.js core modules. The script effectively fetches system details, processes the data, and logs it into a structured file. This project provided hands-on experience with Node.js fundamentals.

GitHub Link:

<https://github.com/VRAJPATEL-07>

Practical 4

Project: Basic Static File Server

Objective: Build an HTTP server to serve static files like HTML, CSS, and images.

Tasks:

- Create a simple HTML page with a welcome message and a few images.
- Use the http and fs modules to serve these files when the browser requests them.
- Add logic to handle 404 errors when a file is not found.

Screenshots of Code:

```
Practical-4 > static-server > public > <> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Welcome to My Static Server</title>
7      <link rel="stylesheet" href="styles.css">
8  </head>
9  <body>
10     <h1>🚀 Welcome to My Static File Server</h1>
11     <p>This is a simple Node.js server serving static files.</p>
12     
13 </body>
14 </html>
```

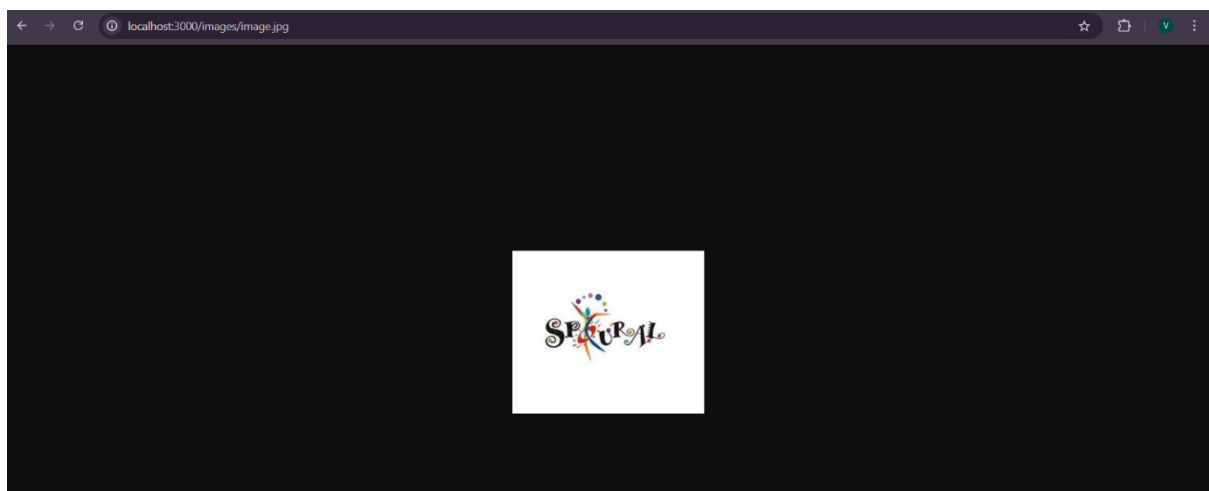
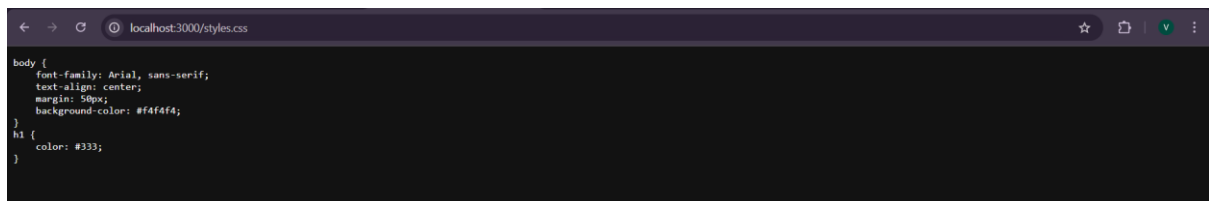
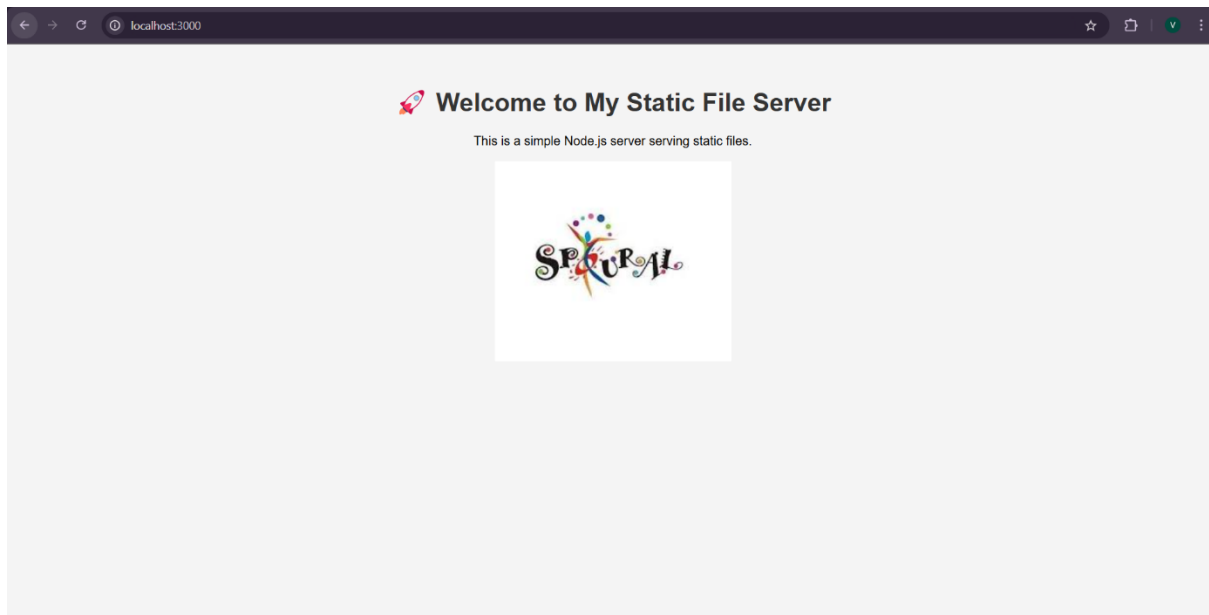
```
Practical-4 > static-server > public > # styles.css > ...
1  body {
2      font-family: Arial, sans-serif;
3      text-align: center;
4      margin: 50px;
5      background-color: #f4f4f4;
6  }
7  h1 {
8      color: #333;
9  }
```

```
Practical-4 > static-server > JS server.js > ...
1  const http = require("http");
2  const fs = require("fs");
3  const path = require("path");
4  const { serveFile } = require("./fileHandler");
5
6  const PORT = 3000;
7  const PUBLIC_DIR = path.join(__dirname, "public");
8
9  const server = http.createServer((req, res) => {
10     let filePath = path.join(PUBLIC_DIR, req.url === "/" ? "index.html" : req.url);
11
12     if (!fs.existsSync(filePath) || fs.lstatSync(filePath).isDirectory()) {
13         res.writeHead(404, { "Content-Type": "text/html" });
14         return res.end("<h1>404 - Page Not Found</h1>");
15     }
16
17     serveFile(filePath, res);
18 });
19
20 server.listen(PORT, () => {
21     console.log(`🔥 Server running at http://localhost:${PORT}/`);
22 });
```

```
Practical-4 > static-server > JS fileHandler.js > ...
1  const fs = require("fs");
2  const path = require("path");
3  const mime = require("mime-types");
4
5  const serveFile = (filePath, response) => {
6     fs.readFile(filePath, (err, data) => {
7         if (err) {
8             response.writeHead(404, { "Content-Type": "text/html" });
9             response.end("<h1>404 - File Not Found</h1>");
10        } else {
11            const contentType = mime.lookup(filePath) || "application/octet-stream";
12            response.writeHead(200, { "Content-Type": contentType });
13            response.end(data);
14        }
15    });
16 };
17
18 module.exports = { serveFile };
```

Screenshots of Output:

```
PS C:\Users\Admin\Documents\full stack\Practicals\practical-4\static-server> node server.js
🔥 Server running at http://localhost:3000/
```



404 - Page Not Found

Conclusion:

In this project, we successfully built a static file server using Node.js core modules. The server serves HTML, CSS, and images, handles different URL paths, and returns a 404 error for missing files.

GitHub Link:

<https://github.com/VRAJPATEL-07>

Practical 5

Project: File Organizer Tool

Objective: Build a CLI tool to organize files into folders based on their type (e.g., images, documents, videos).

Tasks:

- Accept a directory path as an input from the user.
- Use the fs module to read all files in the directory.
- Move files into folders like Images, Documents, and Others based on their extensions.
- Log the operations performed into a summary.txt file.

Screenshots of Code:

Screenshots of Output:

Conclusion:

In this lab, we explored fundamental JavaScript concepts, including variables, functions, arrays, objects, and ES6+ features. By implementing the Movie Collection Manager, we practiced applying these concepts in a structured and practical manner.

GitHub Link:

<https://github.com/VRAJPATEL-07>

